

Functional programming and control

Block 2 — the formalism: the λ -calculus

SW7 · Aalborg University Copenhagen · Autumn 2026

The calculus

Block 1's answers in one formalism: the smallest language that computes everything, and the one language semantics is written in

Syntax: Three forms

x a **variable**

$\lambda x. t$ a **function** of one argument x , with body t

$t u$ an **application** of t to u

application groups to the left

$$f\ a\ b = (f\ a)\ b$$

a body extends as far right as it can

$$\lambda x. x\ y = \lambda x. (x\ y)$$

currying: a function of two arguments is a function returning a function (session 5)

$$\lambda x. \lambda y. t = \lambda x\ y. t$$

No numbers, no booleans, no `let`, no `if`. Everything else is **defined** from these three forms.

Note: In JavaScript the three forms are x , $x \Rightarrow e$ and $f(a)$: $\lambda x. 2 + x$ reads $x \Rightarrow 2 + x$ — except that 2 and $+$ are not in the calculus.

Free and bound variables

In `λx. x y`, `x` is **bound** by the `λ`; `y` is **free**: nothing in the term says what it is. A term with no free variable is **closed**.

```
free (x)      = {x}
free (λx. t)   = free t - {x}
free (t u)     = free t ∪ free u
```

λ-CALCULUS

These three lines are the whole definition; the lab's warm-up writes them in OCaml.

Note: The same name can be bound in one place and free in another: in `x (λx. x)`, the first `x` is free.

β -reduction

$t[x := u]$ is t with u put in for every free x — **substitution**. One rule:
 $(\lambda x. t) u \rightarrow t[x := u]$: the body, with the argument substituted for the variable.

An application of a λ is a **redex**. A term with no redex is in **normal form**: the computation is over. $t \rightarrow^* u$: u is reached from t in some number of steps.

0 $(\lambda x y. x y) (\lambda z. z z) (\lambda w. w)$

Step

Run

Reset

In the trace, β is the rule; δ unfolds a name such as Y into its definition — a courtesy, not a rule.

Substitution and capture

Substitution is defined case by case:

λ-CALCULUS

$$\begin{aligned}x[x := u] &= u \\y[x := u] &= y && \text{if } y \neq x \\(t \ s)[x := u] &= t[x := u] \ s[x := u] \\(\lambda y. t)[x := u] &= \lambda y. t[x := u] && \text{if } y \neq x, y \text{ not free in } u\end{aligned}$$

The last case carries a condition, and the condition matters: in $(\lambda x. \lambda y. x) \ y$, putting y in for x without it gives $\lambda y. y$ — the free y has been **captured** by the λy , and the meaning has changed.

The solution: a bound variable can be renamed without changing the meaning of the term: **α-conversion**. Rename the binder first, then substitute: $(\lambda x. \lambda y'. x) \ y \rightarrow \lambda y'. y$.

Which redex first?

This term has three redexes, so the rule allows three different first steps. **Normal order** takes the outermost one, the leftmost among those; **applicative order** takes the innermost.

normal order

```
0 (λx y. x) ((λz. z) a) ((λz. z) b)
```

Step

Run

Reset

☐ applicative order ☒ normal order

applicative order

```
0 (λx y. x) ((λz. z) a) ((λz. z) b)
```

Step

Run

Reset

☒ applicative order ☐ normal order

Church and Rosser, 1936: a term has **at most one** normal form. A path may fail to reach it — the next slide — but no path reaches a different one. The order changes the path, and whether we arrive; never the answer.

Does the order matter?

$\Omega = (\lambda x. x x) (\lambda x. x x)$ reduces to itself, forever. Apply a function that ignores its argument to it:

normal order

```
0 (λx y. y) Ω
```

Step

Run

Reset

☐ applicative order ☒ normal order

applicative order

```
0 (λx y. y) Ω
```

Step

Run

Reset

☒ applicative order ☐ normal order

The order never changes the answer; it decides **whether we get one**. Normal order finds the normal form whenever one exists.

Reduction orders and parameter-passing modes

reduction order	the mode, block 1	languages
applicative: arguments first	call-by-value (weak)	C, Java, Python, JavaScript, OCaml, Scheme
normal: arguments when used, each time	call-by-name (weak)	Algol 60 — its default
normal, with sharing: each argument once	call-by-need	Haskell

Evaluation strategy: a design decision every language has made, mostly the same way.

Programming languages evaluate **weakly**: a function body runs only when the function is called, never inside a λ . So does the lab's evaluator; the stepper has a switch.

Small-step operational semantics: the rules

call by value

$$\frac{t_1 \rightarrow t'_1}{t_1 t_2 \rightarrow t'_1 t_2}$$

E-APP1

$$\frac{t_2 \rightarrow t'_2}{v t_2 \rightarrow v t'_2}$$

E-APP2

$$(\lambda x. t) v \rightarrow t[x := v]$$

E-APPABS

First the function part is reduced, then the argument, until each is a value v — a λ ; only then does β apply.

call by name

$$\frac{t_1 \rightarrow t'_1}{t_1 t_2 \rightarrow t'_1 t_2}$$

E-APP1

$$(\lambda x. t_1) t_2 \rightarrow t_1[x := t_2]$$

E-APPABS

Only the function part is reduced, until it is a λ ; then β applies with the argument still unevaluated.

No rule reduces inside a λ : both are weak. The lab's `step_cbv` and `step_cbn` are these rules, one `match` case each.

Everything from functions

Booleans, numbers, if, recursion — defined, not built in

Booleans are choices

λ-CALCULUS

```
true  = λt. λf. t
false = λt. λf. f

if = λc. λt. λe. c t e
```

A boolean is a function that, given two things, picks one. `if` only hands the two branches to the condition.

```
0 if true a b
```

Step

Run

Reset

☐ applicative order

☒ normal order

☐ weak (never inside λ)

☐ names

`if` is a function — under normal order.

if as a function under call-by-value

Block 1's `my_if` failed: both branches are evaluated before `if` runs. Two repairs — the language evaluates by name, or the branches are delayed by hand:

```
fact  = λf n. if (iszero n) 1 (mul n (f (pred n)))
factv = λf n. if (iszero n) (λd. 1) (λd. mul n (f (pred n))) I
Z      = a term that supplies the recursive call — defined three slides on
```

λ-CALCULUS

0 Z fact 2

0 Z factv 2

Step **Run** **Reset** ☒ applicative order ☐ normal order

Never ends: both branches are evaluated before `if` chooses.

Step **Run** **Reset** ☒ applicative order ☐ normal order

Ends, with 2: only the chosen branch is applied — a **thunk**, block 1's `fun () ->`.

Here, the branches delayed by hand; on the previous slide, the language evaluating by name.

Numbers are repetitions

The number n is the function that applies `f` to `x` n times; `succ` adds one more application, `add` chains them.

```
0 = λf x. x           1 = λf x. f x           3 = λf x. f (f (f x))
succ = λn f x. f (n f x)  add = λm n f x. m f (n f x)
```

Λ-CALCULUS

```
0 add 2 1
```

Step **Run** **Reset** ☐ applicative order ☒ normal order ☐ weak (never inside λ) ☒ names

A number applies a function n times: **a number is its own fold** — block 1's *The body as a parameter*, literally.

Recursion without `let rec`

The calculus has no `let rec`, so `fact` cannot call itself by name. Instead it takes the function to call as an argument `f`:

`fact = λf n. if (iszero n) 1 (mul n (f (pred n)))`. Recursion then needs a term `fix` that hands a function itself: `fix g = g (fix g)`.

$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$

$Y g \rightarrow^* g (Y g)$

λ-CALCULUS

0 Y fact 3

Step

Run

Reset

☐ applicative order

☒ normal order

☐ weak (never inside λ)

☒ names

Y under call-by-value

PYTHON

```
Y = lambda f: (lambda x: f(x(x)))(lambda x: f(x(x)))
Z = lambda f: (lambda x: f(lambda v: x(x)(v)))(lambda x: f(lambda v: x(x)(v)))
fact = lambda self: lambda n: 1 if n == 0 else n * self(n - 1)
print(Z(fact)(5))
print(Y(fact)(5))
```

Run

120

live

```
File "<exec>", line 1, in <lambda>
File "<exec>", line 1, in <lambda>
[Previous line repeated 992 more times]
RecursionError: maximum recursion depth exceeded
```

Y **g** evaluates the argument **x x** first — forever.
Z wraps it in a thunk, **λv. x x v**: the repair of *if* under call-by-value again.

```
# let y f = (fun x -> f (x x)) (fun x -> f (x x)) recorded
Error: The value x has type 'a -> 'b
      but an expression was expected of type 'a
      The type variable 'a occurs inside 'a -> 'b
```

OCaml refuses **Y**'s type before it can run it — a problem on top of the order problem; session 10.

Laziness: by-name with sharing

Call-by-name evaluates an argument every time it is used. **Call-by-need** evaluates it once, when first used, and remembers the result.

```
let x = lazy (print_string "once "; 6 * 7);;  
Lazy.force x;;  
Lazy.force x;;
```

OCAML

```
once - : int = 42  
- : int = 42
```

recorded

When the *language* decides laziness, an infinite list is a value. Session 5: when the *program* decides it — thunks and streams in a strict language.

```
ghci> (\x -> 1) undefined
```

HASKELL

```
1
```

```
ghci> fibs = 0:1:zipWith (+) fibs (tail fibs)
```

```
ghci> take 10 fibs
```

```
[0,1,1,2,3,5,8,13,21,34]
```

An argument never used is never evaluated; a list with no end is a value.

Evaluation strategies, stated

The three strategies, side by side: when an argument is evaluated, and what each choice costs.

strategy	an argument is evaluated	what it costs
call-by-value	before the call, once	work on arguments never used; Ω is fatal
call-by-name	at each use	the same argument evaluated many times
call-by-need	at first use, once	the order of effects is no longer readable from the program

That last cost is why Haskell keeps effects out of ordinary expressions — a consequence of laziness.

Control as syntax, control as a value

- no syntax The calculus has no `;`, no `while`, no `if`. The text says *where* a step may happen, never *which*: the order is not in the program.
- values So control lives in values, and the values are functions: a boolean **picks**, a numeral **repeats**, `Y` **recurses**, a thunk **delays**, a continuation **continues**. This part, and session 5.
- syntax again A functional language keeps some control syntax — `if`, `&&`, `let ... in`, `;`. Each fixes an order the calculus left open. Block 1's `my_if` is `if` with the syntax taken away.

Functions as values are not a feature the calculus happens to have. They are what a language without control syntax **must** have — and they suffice.

Conclusion

The first answer, and the lab

Who decides the next step — after session 4

session	topic	who decides the next step
S4	evaluation	the language: whether a call keeps a frame, in what order the arguments go — and when an argument is evaluated — its evaluation strategy
S5	higher-order functions	the program, by passing the next step as a function: a loop body, a thunk, a continuation
S6	async	the program yields; an event loop decides
S7	threads	the operating system — preemption
S8	message passing	the other process — communication
S9	parallelism	nobody needs to — the answer is order-independent

Lab 4: an evaluator for the calculus

```
type term =  
  | Var of string  
  | Lam of string * term  
  | App of term * term  
  
type value =  
  | Closure of string * term * env  
and env = (string * value) list
```

OCAML

warm-up reductions on paper; `free_vars`, `subst`
without capture

A `step` under call-by-value and call-by-name, with a trace: `Ω` and `(λx y. y) Ω`
in our own code

B a big-step evaluator with environments: a
function value is a **closure**

home Church arithmetic as terms; `Y` and `Z` in
our evaluator; a small ML

What does the closure carry? — session 5's first
question.

References

- Turing, 1936–37. On computable numbers, with an application to the Entscheidungsproblem. Church, 1936. An unsolvable problem of elementary number theory. Kleene, 1936.
- Church and Rosser, 1936. Some properties of conversion. Curry and Feys, 1958. Barendregt, 1984. The Lambda Calculus.
- Plotkin, 1975. Call-by-name, call-by-value and the λ -calculus. Wadsworth, 1971. Ariola and Felleisen, 1997. The call-by-need lambda calculus.
- Landin, 1964. The mechanical evaluation of expressions. Reynolds, 1972. Definitional interpreters for higher-order programming languages.
- Steele, 1977. Debunking the "expensive procedure call" myth. Clinger, 1998. Proper tail recursion and space efficiency.
- Wirth, 1976. Algorithms + Data Structures = Programs. Kowalski, 1979. Algorithm = Logic + Control. Backus, 1978. Can programming be liberated from the von Neumann style?
- Böhm and Berarducci, 1985. Automatic synthesis of typed Λ -programs on term algebras. Hughes, 1989. Why functional programming matters.